Experiment 4

Adders, Two's-Complement Overflow and Hamming Error-Correcting Codes

Student	Seyed Kiarash Eshraghi Naeini (403101383)
Instructor	Dr. Mohammad Reza Movahhedin Teaching assistant: Hossein Moghim
Semester	Fall 2025 session: 16–17 November 2025
Platform	Zynq-7010 (XC7Z010-CLG400-1), 50 MHz clock, Vivado 2019.1

Summary

The first part of this experiment builds arithmetic from the bottom up: a seven-segment decoder, a 3-bit ripple-carry adder made of full adders (themselves made of half adders), and a 4-bit two's-complement adder/subtractor with overflow detection and a sign-magnitude display. The second part implements the Hamming(7,4) single-error-correcting code: encoder, decoder, a noisy "transmission line" controlled from the switches and buttons, and finally the extended (8,4) SECDED code that also detects double errors. All seven designs were verified exhaustively for this report (7/7 pass), including every single- and double-error pattern for every data word. Code: `https://github.com/seyedkiarash/seyedkiarash.github.io/tree/main/labs/logic-circuits-lab/lab04-adders-hamming`.

1 Part 1 – Hexadecimal display

`Hex_to_7_seg` decodes `DIP[3:0]` into the segment pattern of `0`–`F`; the segments of the extension board are active-low, so the positive-logic patterns are inverted. The module was also packaged as an IP and instantiated in an IP-Integrator block design. Test: 256 DIP settings, 0 errors.

2 Part 2 – 3-bit ripple-carry adder

A half adder gives $s = a \oplus b$, $c = ab$; two half adders and an OR gate form a full adder with $c_{\text{out}} = ab + c_{\text{in}}(a \oplus b)$; three full adders chained through their carries form the 3-bit adder. With $A =$ `DIP[3:1]`, $B =$ `DIP[6:4]` and $c_{\text{in}} =$ `DIP[7]`, the 4-bit result $\{c_{\text{out}}, S\}$ (0–15) is shown in hexadecimal. Test: all 256 combinations, 0 errors.

3 Part 3 – Adder/subtractor with overflow

3.1 Design

For subtraction, $A - B = A + \overline{B} + 1$: every bit of B passes through an XOR with the operation signal `Op`, which is also the carry-in of the first stage. With 4-bit two's-complement operands (-8...7) the result overflows when the carry *into* the sign bit differs from the carry *out of* it, $V = c_3 \oplus c_4$. The result is converted to sign and magnitude for display: the magnitude appears on the seven-segment digit, `LED[9]` shows a minus sign and `LED[0]` signals overflow – in which case the digit is switched off (Fig. 1).



Figure 1. Part 3 display for a positive result, a negative result and an overflow.

3.2 Verification

All 256×2 operand/operation combinations were compared with integer arithmetic: the 128 results outside $-8 \ldots 7$ are flagged as overflow with the digit blanked, and all 384 others show the correct magnitude and

sign. 0 errors.

4 Parts 4–5 – Hamming(7,4) encoder and decoder

4.1 Code construction

The four data bits $x_0 \ldots x_3$ and three parity bits p_0, p_1, p_2 occupy the positions 1–7 of the code word so that each parity bit sits at a power-of-two position and covers the positions whose index has the corresponding bit set (Table 1). With even parity

$$p_0 = x_0 \oplus x_1 \oplus x_3, \qquad p_1 = x_0 \oplus x_2 \oplus x_3, \qquad p_2 = x_1 \oplus x_2 \oplus x_3.$$

Table 1. Hamming(7,4) layout; • marks the positions checked by each parity bit.

Position	1	2	3	4	5	6	7
Bit	p_0	p_1	x_0	p_2	x_1	x_2	x_3
p_0 (bit 0 of position)	•		•		•		•
p_1 (bit 1)		•	•			•	•
p_2 (bit 2)				•	•	•	•

4.2 Decoding

The decoder recomputes the three checks, $c_0 = p_0 \oplus x_0 \oplus x_1 \oplus x_3$ and so on. Because each position is covered by exactly the checks named by its binary index, the syndrome $\{c_2, c_1, c_0\}$ *is* the position of a single flipped bit (0: no error). Data bits at positions 3, 5, 6 and 7 are inverted when the syndrome points at them; the syndrome is shown on the seven-segment display.

4.3 Verification

Encoder: for all 16 data words the three parity checks of the code word are even and the data bits are in place. Decoder: every data word with no error and with each of the 7 single-bit errors (128 cases) – the data is recovered and the displayed position equals the flipped position.

5 Part 6 – Encoder, noisy channel and decoder

The encoder output passes through a "transmission line" that XORs it with a 7-bit noise vector set by `DIP[7:5]` and the four (active-low) buttons; the decoder output is shown on `LED[4:1]`. Every data word was tested with every noise pattern of weight 0, 1 and 2:

- all **128** cases with no or one error are corrected;
- all **336** double-error cases are decoded *wrongly*. This is the expected limitation of a distance-3 code: two errors produce a non-zero syndrome that points at a third, correct bit, so the decoder "corrects" it and outputs a wrong word without any warning.

6 Part 7 – Extended Hamming (8,4) SECDED

An eighth bit, the parity of the whole 7-bit code word, raises the minimum distance to 4. The decoder combines the syndrome s with the overall parity check P:

s	P	Interpretation	Output
0	even	no error	data, display `0`
$\neq 0$	odd	single error at position s	corrected data, display s
0	odd	the overall parity bit flipped	data, display `0`
$\neq 0$	even	**double error** – detected, not correctable	`EE` on two digits, `LED[4:1]` = `1111`



Figure 2. Part 7 display: no error, a corrected single error and a detected double error.

Verification over all data words and all noise patterns of weight ≤ 2 on the 8-bit word: the **144** cases with no or one error are corrected and all **448** double-error cases are detected. Compared with Part 6, the extra parity bit turns 336 silent miscorrections into 448 reported errors.

7 Implementation results

All designs of this experiment are purely combinational (no flip-flops, no clock constraint). The SECDED design of Part 7 uses **17 LUTs** in the Vivado implementation of the lab session (33 I/O pins); Yosys synthesis of the other parts gives 3–20 LUTs each.

8 Verification summary

Testbench	What is checked (exhaustive)	Result
`tb_part1`	256 DIP settings against a reference decoder	**PASS**
`tb_part2`	all 256 (A, B, c_{in}) combinations	**PASS**
`tb_part3`	512 cases, overflow, sign and magnitude	**PASS**
`tb_part4`	16 code words: parity checks and data positions	**PASS**
`tb_part5`	16 words × (no error + 7 single errors)	**PASS**
`tb_part6`	16 words × all noise patterns of weight ≤ 2	**PASS**
`tb_part7`	16 words × all 8-bit noise patterns of weight ≤ 2	**PASS**

No functional changes were needed in this experiment; the cleaned code shares the half/full-adder cells between the parts and has English comments.

9 Conclusion

Hierarchical design – half adder, full adder, ripple-carry adder, adder/subtractor – keeps each module trivial while the whole performs signed arithmetic with correct overflow detection. The Hamming experiments show both sides of a distance-3 code: every single error is corrected, but double errors are silently miscorrected; one extra parity bit (SECDED) converts all of those into detected errors.